

CEFET-MG
ALGORITMOS E ESTRUTURAS DE DADOS II

Matheus Tavares de Bessas

Trabalho Prático Individual I: Estruturas em
Árvores Avançadas

Divinópolis - MG
Agosto, 2026

SUMÁRIO

1	INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA	3
1.1	Contextualização e Motivação	3
1.2	Estruturas Analisadas e Propriedades Fundamentais	4
1.3	Trie (Árvore de Prefixos)	4
1.4	Árvore Patricia (Radix Tree Compacta)	4
1.5	Árvore Splay	5
1.6	Árvore Treap (Tree + Heap)	5
1.7	KD-Tree (k-dimensional Tree)	5
1.8	Matriz Comparativa Conceitual	6
2	PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS	8
2.1	Representação Computacional e Decisões de Projeto	8
2.2	Trie (Árvore de Prefixos)	8
2.3	Árvore Patricia (Radix Tree Compacta)	8
2.4	Árvore Splay	9
2.5	Árvore Treap	10
2.6	KD-Tree (k-dimensional Tree)	10
3	DEMONSTRAÇÃO E RASTREAMENTO VISUAL	12
3.1	Demonstração da Trie	14
3.2	Demonstração da Árvore Patricia	15
3.3	Demonstração da Árvore Splay	16
3.4	Demonstração da Árvore Treap	17
3.5	Demonstração da KD-Tree	18
4	ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA . . .	19
4.0.1	Tabela Comparativa de Complexidade Assintótica	19
5	ANÁLISE EXPERIMENTAL E RESULTADOS	20
5.1	Ambiente de Execução e Metodologia	20
5.2	Análise de Tempo de Inserção: Trie vs. Árvore Patricia	20
5.3	Análise da Quantidade de Comparações: Trie vs. Árvore Patricia . .	21
5.4	Análise de Rotações: Splay Tree vs. Treap	22
6	APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO	24
6.1	Aplicações Práticas e Adequação Estrutural	24

6.1.1	1. Trie: Autocompletar e Sugestões de Pesquisa em Tempo Real	24
6.1.2	2. Árvore Patricia: Tabela de Roteamento IP (LPM – Longest Prefix Match)	24
6.1.3	3. Splay Tree: Cache de Memória e Listas de Arquivos Recentes (LRU/Temporal)	24
6.1.4	4. Treap: Tabelas de Símbolos Probabilísticas e Conjuntos Ordenados Dinâmicos	25
6.1.5	5. KD-Tree: Sistemas de Informação Geográfica (GIS) e Classificador k -NN	25
6.2	Análise Crítica: Vantagens e Limitações Observadas	25
6.2.1	Desafios de Implementação e Esforço de Desenvolvimento	26
6.2.2	Situações de Superioridade Prática e Melhorias Futuras	26
6.2.3	Propostas de Extensões e Melhorias	27
7	CONCLUSÃO	28
7.0.1	Diferenças Fundamentais e Filosofia de Projeto	28
7.0.2	Desempenho Relativo por Operação e Conjunto de Dados	28
7.0.3	Coerência entre Resultados Experimentais e Complexidade Teórica	29
7.0.4	Dificuldades de Implementação e Esforço de Desenvolvimento	29
7.0.5	Cenários de Aplicação Ideal	29
7.0.6	Síntese Crítica de Vantagens e Limitações	30

1 INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

1.1 CONTEXTUALIZAÇÃO E MOTIVAÇÃO

As estruturas de dados hierárquicas desempenham um papel central na Ciência da Computação, sendo fundamentais para a organização, indexação e recuperação eficiente de informações em larga escala. Estruturas clássicas como as Árvore Binárias de Busca (*Binary Search Trees* – BST) e as Árvore Balanceadas de Adelson-Velsky e Landis (AVL) oferecem garantias teóricas importantes, operando com complexidade de tempo de busca em $\mathcal{O}(\log n)$ no caso médio e $\mathcal{O}(\log n)$ estrito no pior caso para a AVL.

No entanto, em cenários do mundo real, o desempenho absoluto dessas estruturas pode ser limitado devido a premissas simplificadas:

1. **Natureza dos Dados e Comparação de Chaves:** BSTs e AVLs tratam as chaves como instâncias atômicas, realizando comparações completas a cada nó. Para chaves compostas (como cadeias de caracteres longas ou URLs), o custo real de uma única comparação é proporcional ao tamanho do prefixo comum, degradando o tempo de busca prático.
2. **Localidade de Referência Temporal:** Árvore com balanceamento rígido (como a AVL) tratam todas as consultas com a mesma prioridade espacial. Em aplicações onde um pequeno subconjunto de dados é acessado com frequência desproporcional (padrão 80/20 ou *cache-like*), o custo fixo de $\mathcal{O}(\log n)$ em todas as buscas torna-se subótimo.
3. **Complexidade Dimensional:** A ordenação total 1D imposta pelas BSTs e AVLs falha em representar dados multidimensionais (como coordenadas espaciais em $\mathbb{R}^k$), tornando inviável a realização eficiente de consultas por faixa (*range search*) ou de vizinhos mais próximos (*nearest neighbors*) sem incorrer em uma busca exaustiva de ordem $\mathcal{O}(n)$.

Diante dessas limitações, surge a necessidade de estudar estruturas de dados hierárquicas especializadas. Este trabalho aborda a modelagem, implementação e análise de cinco estruturas alternativas: **Trie**, **Árvore Patricia**, **Árvore Splay**, **Árvore Treap** e **KD-Tree**.

1.2 ESTRUTURAS ANALISADAS E PROPRIEDADES FUNDAMENTAIS

1.3 TRIE (ÁRVORE DE PREFIXOS)

A Trie é uma árvore m -ária projetada para a recuperação de sequências de caracteres, na qual a chave não é armazenada no próprio nó, mas sim determinada pela sua posição relativa na hierarquia.

- **Propriedades e Invariantes:** Nós compartilhados representam prefixos comuns entre diferentes chaves. Se duas palavras compartilham os primeiros k caracteres, elas utilizam o mesmo caminho até o k -ésimo nó.
- **Anatomia do Nó:** Contém uma tabela de ponteiros/referências para os filhos (tipicamente um *array* ou mapa de espalhamento indexado pelo alfabeto Σ) e uma flag booleana (*is_end_of_word*) indicando se aquele nó encerra uma chave válida.
- **Problema Alvo:** Buscas textuais e recuperação rápida de chaves onde o tempo depende estritamente do comprimento da chave k , e não da quantidade total de elementos n .

1.4 ÁRVORE PATRICIA (RADIX TREE COMPACTA)

A Árvore Patricia é uma variação otimizada da Trie que elimina nós unidirecionais encadeados sem bifurcações (de grau de saída 1), realizando a compressão de caminhos (*path compression*).

- **Propriedades e Invariantes:** Todo nó interno da Patricia possui pelo menos dois filhos (grau de saída ≥ 2). As arestas/nós não contêm caracteres isolados, mas sim *substrings* representando prefixos completos.
- **Anatomia do Nó:** Armazena o trecho do prefixo (*string* ou fatia de bits), o dicionário de filhos e a flag de fim de chave (*is_end_of_word*).
- **Problema Alvo:** Mitigar o alto consumo de memória inerente à Trie original, mantendo a eficiência de buscas por prefixo em tabelas de roteamento IP e indexação compacta.

1.5 ÁRVORE SPLAY

A Árvore Splay é uma árvore binária de busca autoajustável (*self-adjusting*) que utiliza operações de rotação para mover qualquer nó consultado, inserido ou removido para a raiz da árvore.

- **Propriedades e Invariantes:** Mantém a propriedade fundamental de uma BST (subárvore esquerda $<$ nó $<$ subárvore direita), mas não impõe um critério estrito de balanceamento de altura como a AVL. Reorganiza-se dinamicamente através do mecanismo de *Splaying*, que aplica combinações de rotações *Zig*, *Zig-Zig* e *Zig-Zag*.
- **Anatomia do Nó:** Similar à de uma BST padrão, contendo o valor da chave (**key**), e ponteiros para os filhos esquerdo e direito (**left**, **right**).
- **Problema Alvo:** Otimização do tempo de acesso em sistemas com localidade temporal de referência, garantindo um custo amortizado de $\mathcal{O}(\log n)$ por operação sem necessidade de armazenar informações extras de altura ou fator de balanceamento.

1.6 ÁRVORE TREAP (TREE + HEAP)

A Treap é uma estrutura híbrida de busca binária probabilística que combina as propriedades de uma BST e de um Max-Heap.

- **Propriedades e Invariantes:** Cada nó possui um par (chave, prioridade). A chave obedece rigorosamente à propriedade de BST em ordem simétrica. A prioridade (atribuída aleatoriamente no momento da inserção) obedece à propriedade de Max-Heap, onde a prioridade do nó pai é estritamente maior ou igual à de seus filhos.
- **Anatomia do Nó:** Contém a chave principal (**key**), um inteiro de prioridade gerado pseudo-aleatoriamente (**priority**), e referências para os filhos **left** e **right**.
- **Problema Alvo:** Prover um balanceamento probabilístico de altura com altíssima probabilidade de resultar em altura $\mathcal{O}(\log n)$, evitando casos de degradação sem a complexidade de algoritmos de rebalanceamento determinísticos.

1.7 KD-TREE (K-DIMENSIONAL TREE)

A KD-Tree é uma árvore binária de busca espacial projetada para particionar pontos em um espaço euclidiano de dimensão k ($\mathbb{R}^k$).

- **Propriedades e Invariantes:** A árvore alterna ciclicamente o eixo de corte (*hyperplane*) a cada nível da hierarquia ($d = \text{profundidade} \pmod k$). Um nó em uma

profundidade que corta pelo eixo x dividirá o espaço em duas partes: os pontos à esquerda possuem a coordenada x menor, e os à direita, maior ou igual.

- **Anatomia do Nó:** Armazena uma tupla/vetor representando o ponto k -dimensional (ex: (x, y) em 2D), o nível/eixo de corte e referências para as subárvores esquerda e direita.
- **Problema Alvo:** Suportar com alta eficiência consultas multidimensionais, como a busca por vizinhos mais próximos (*Nearest Neighbor Search* – NNS) e buscas por alcance (*Range Queries*).

1.8 MATRIZ COMPARATIVA CONCEITUAL

A Tabela 1 resume as características fundamentais e as estratégias adotadas por cada uma das estruturas analisadas em comparação às árvores tradicionais.

Tabela 1 – Matriz comparativa conceitual das estruturas de dados hierárquicas.

Estrutura	Critério de Organização	Requisitos de Memória por Nó	Estratégia de Balanceamento / Reorganização	Espaço de Chaves Suportado
BST	Ordenação Total 1D ($<, >, =$)	Baixo (Chave + 2 ponteiros)	Nenhum (sujeito à degradação $\mathcal{O}(n)$)	Unidimensional ordenável
AVL	Ordenação Total 1D ($<, >, =$)	Médio (Chave + 2 ponteiros + Fator de Balanceamento)	Estrito determinístico por rotações	Unidimensional ordenável
Trie	Decomposição por caracteres da chave	Alto (Vetor de ponteiros $ \Sigma $ + flag)	Estático pela profundidade do prefixo	Sequências / <i>Strings</i>
Patricia	Compressão de prefixos compartilhados	Médio (Prefix String + Dicionário de filhos)	Compressão / fusão estrutural de caminhos	Sequências / <i>Strings</i>
Splay	Ordenação Total 1D + Frequência de Acesso	Baixo (Chave + 2 ponteiros)	Autoajustável dinâmico por <i>Splaying</i> amortizado	Unidimensional ordenável
Treap	Ordenação (Chave) + Max-Heap (Prioridade)	Médio (Chave + Prioridade + 2 ponteiros)	Probabilístico via rotações na inserção/remoção	Unidimensional ordenável
KD-Tree	Particionamento espacial por eixos alternados	Baixo (Tupla k -D + 2 ponteiros)	Particionamento cíclico ortogonal por dimensão	Multidimensional ($\mathbb{R}^k$)

2 PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

2.1 REPRESENTAÇÃO COMPUTACIONAL E DECISÕES DE PROJETO

Nesta seção são apresentadas as decisões de projeto e a implementação computacional das cinco estruturas hierárquicas. As estruturas foram desenvolvidas em Python 3 utilizando orientação a objetos para garantir modularidade e reuso de código. O código completo encontra-se disponível no repositório do projeto.

2.2 TRIE (ÁRVORE DE PREFIXOS)

A Trie foi implementada utilizando uma estrutura de nós onde os ponteiros para os filhos são mantidos em um dicionário.

```

1 class TrieNode:
2     def __init__(self):
3         self.children = {} # Mapeia caractere -> TrieNode
4         self.is_end_of_word = False
5
6 class Trie:
7     def __init__(self):
8         self.root = TrieNode()
9
10    def insert(self, word: str) -> None:
11        node = self.root
12        for char in word:
13            if char not in node.children:
14                node.children[char] = TrieNode()
15            node = node.children[char]
16        node.is_end_of_word = True

```

Listing 2.1 – Implementação do nó e inserção na Trie.

2.3 ÁRVORE PATRICIA (RADIX TREE COMPACTA)

A Árvore Patricia realiza a compressão de caminhos unindo nós consecutivos que possuem grau de saída igual a 1.

```

1 class PatriciaNode:
2     def __init__(self, prefix: str = ""):
3         self.prefix = prefix
4         self.children = {} # Mapeia primeiro caractere do filho ->
                             PatriciaNode
5         self.is_end_of_word = False
6
7 class PatriciaTree:
8     def __init__(self):
9         self.root = PatriciaNode()
10
11     def insert(self, word: str) -> None:
12         # Lógica de busca do maior prefixo comum (LCP) e divisão (
13         split) do n
14         pass

```

Listing 2.2 – Estrutura de nó e lógica de divisão de prefixos na Patricia Tree.

2.4 ÁRVORE SPLAY

A Splay Tree reestrutura-se dinamicamente a cada operação através do procedimento de *splaying*, trazendo o nó acessado para a raiz.

```

1 class SplayNode:
2     def __init__(self, key: int):
3         self.key = key
4         self.left = None
5         self.right = None
6
7 class SplayTree:
8     def __init__(self):
9         self.root = None
10
11     def _right_rotate(self, x: SplayNode) -> SplayNode:
12         y = x.left
13         x.left = y.right
14         y.right = x
15         return y
16
17     def splay(self, root: SplayNode, key: int) -> SplayNode:
18         # Aplica o recursiva ou iterativa de rota es Zig, Zig-Zig
19         e Zig-Zag
20         pass

```

Listing 2.3 – Operação de rotação e Splaying na Árvore Splay.

2.5 ÁRVORE TREAP

A Treap combina as propriedades de uma BST (pela chave) e de um Max-Heap (pela prioridade gerada aleatoriamente).

```
1 import random
2
3 class TreapNode:
4     def __init__(self, key: int):
5         self.key = key
6         self.priority = random.randint(1, 100)
7         self.left = None
8         self.right = None
9
10 class Treap:
11     def insert(self, root: TreapNode, key: int) -> TreapNode:
12         if not root:
13             return TreapNode(key)
14         if key < root.key:
15             root.left = self.insert(root.left, key)
16             if root.left.priority > root.priority:
17                 root = self._right_rotate(root)
18         else:
19             root.right = self.insert(root.right, key)
20             if root.right.priority > root.priority:
21                 root = self._left_rotate(root)
22         return root
```

Listing 2.4 – Inserção e rebalanceamento via rotações na Treap.

2.6 KD-TREE (K-DIMENSIONAL TREE)

A KD-Tree alterna o eixo de corte a cada nível para realizar o particionamento do espaço k -dimensional.

```
1 class KNode:
2     def __init__(self, point: tuple, axis: int):
3         self.point = point
4         self.axis = axis
5         self.left = None
6         self.right = None
7
8 class KDTree:
9     def build(self, points: list, depth: int = 0) -> KNode:
10         if not points:
11             return None
12         k = len(points[0])
```

```
13     axis = depth % k
14     points.sort(key=lambda x: x[axis])
15     median = len(points) // 2
16
17     node = KDNode(points[median], axis)
18     node.left = self.build(points[:median], depth + 1)
19     node.right = self.build(points[median + 1:], depth + 1)
20     return node
```

Listing 2.5 – Construção recursiva e alternância de eixos na KD-Tree.

3 DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Nesta seção são apresentadas as representações gráficas das cinco estruturas em três estados distintos de execução: estado inicial, estado intermediário (após uma operação chave) e estado final após remoção/reorganização.

3.1 DEMONSTRAÇÃO DA TRIE

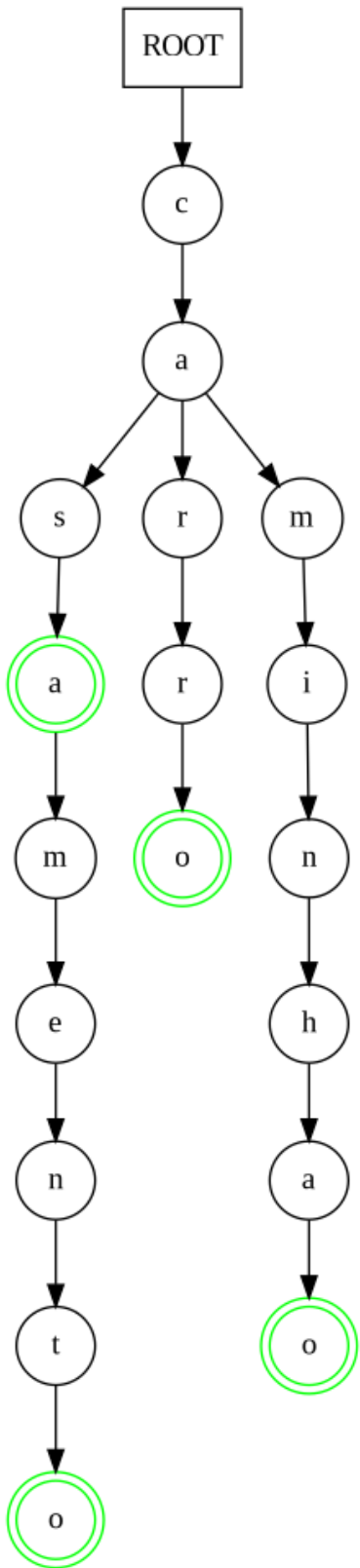


Figura 1 – Estado inicial.

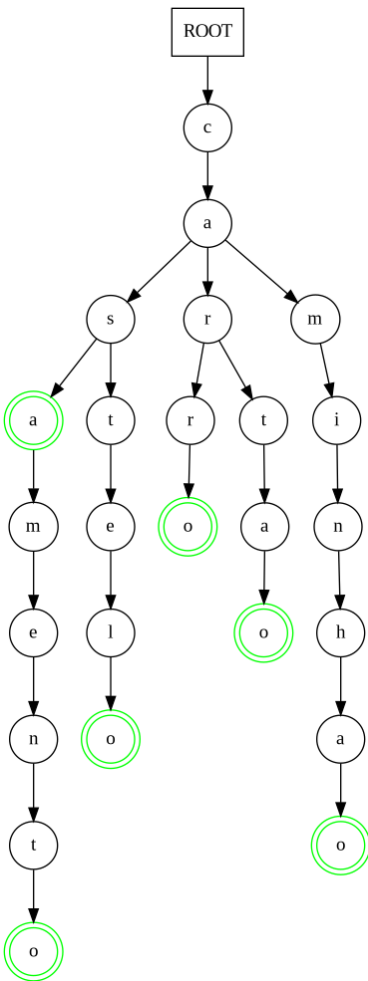


Figura 2 – Estado intermediário.

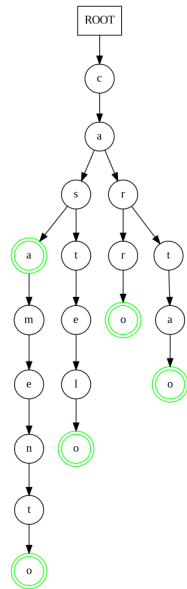


Figura 3 – Pós-remoção.

Figura 4 – Evolução estrutural da Trie durante as operações.

3.2 DEMONSTRAÇÃO DA ÁRVORE PATRICIA

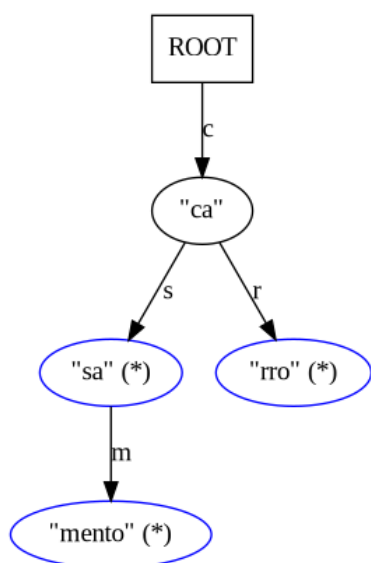


Figura 5 – Estado inicial.

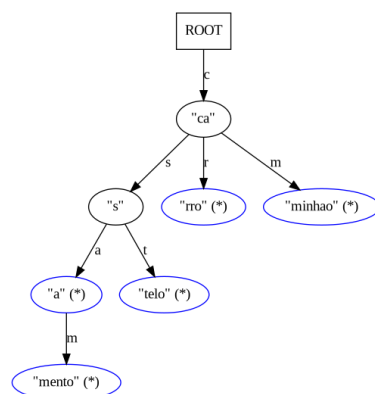


Figura 6 – Estado intermediário

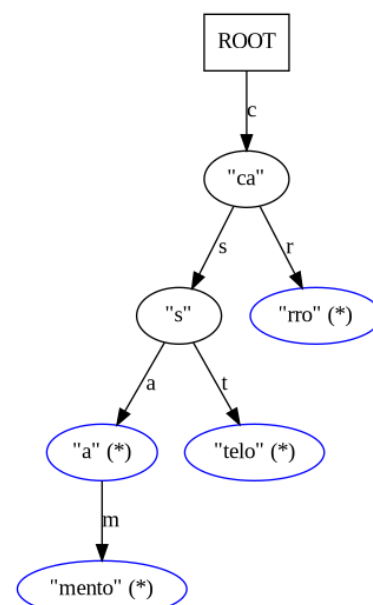


Figura 7 – Pós-remoção.

Figura 8 – Evolução estrutural da Árvore Patricia com compressão de caminhos.

3.3 DEMONSTRAÇÃO DA ÁRVORE SPLAY

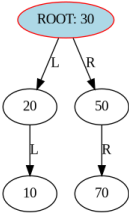


Figura 9 – Estado inicial.

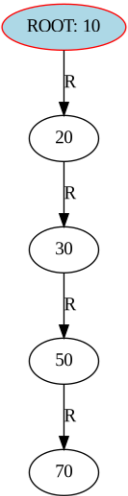


Figura 10 – Estado intermediário.

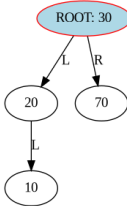


Figura 11 – Pós-remoção.

Figura 12 – Evolução estrutural da Splay Tree destacando a movimentação do nó acessado.

3.4 DEMONSTRAÇÃO DA ÁRVORE TREAP

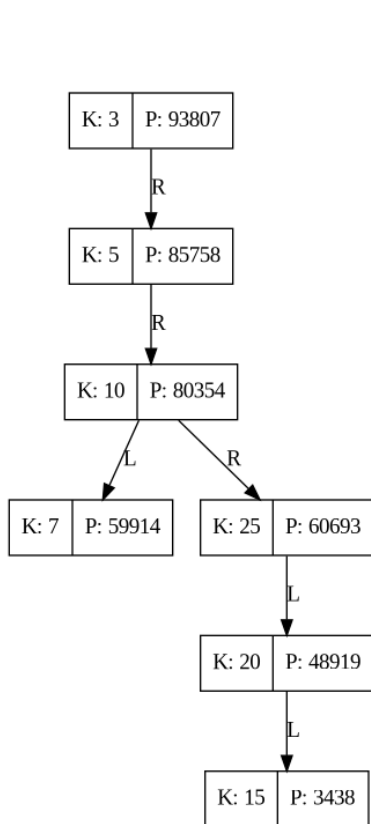


Figura 13 – Estado inicial.

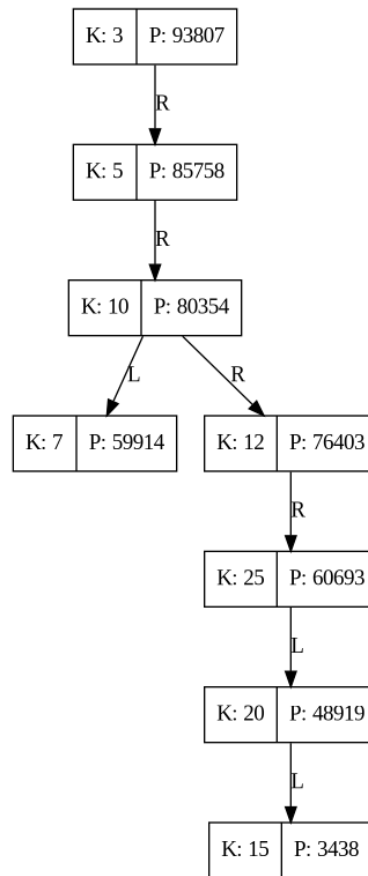


Figura 14 – Estado intermediário.

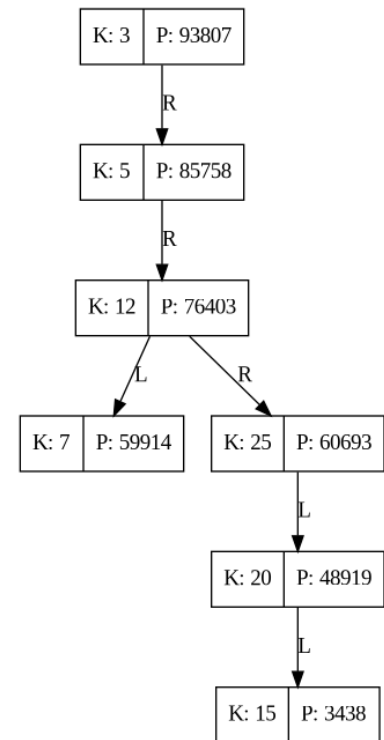


Figura 15 – Pós-remoção.

Figura 16 – Evolução estrutural da Treap com reorganização baseada na prioridade.

3.5 DEMONSTRAÇÃO DA KD-TREE

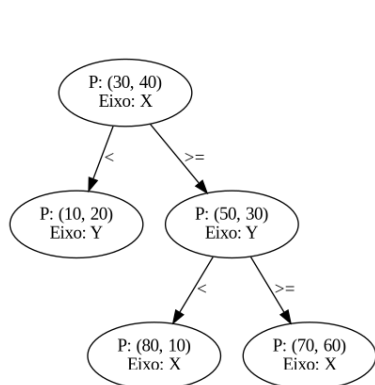


Figura 17 – Particionamento inicial.

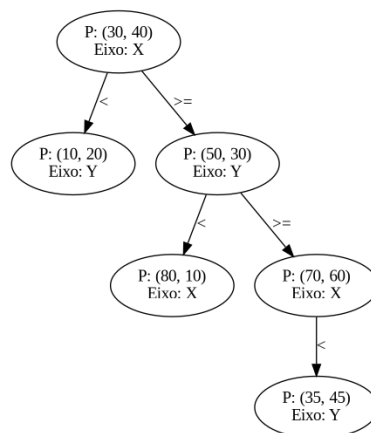


Figura 18 – Estado intermediário.

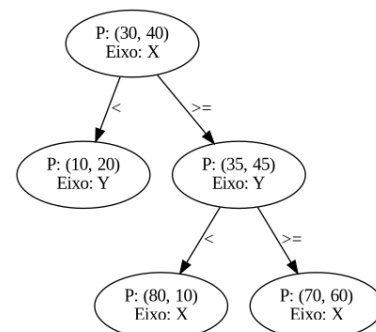


Figura 19 – Pós-remoção.

Figura 20 – Evolução da KD-Tree no espaço 2D e na estrutura de árvore.

4 ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

Nesta seção, apresenta-se a análise formal do custo computacional das operações fundamentais para as cinco estruturas de dados implementadas (*Trie*, *Árvore Patricia*, *Splay Tree*, *Treap* e *KD-Tree*), em comparação com as estruturas clássicas de busca binária: *Binary Search Tree* (BST) não balanceada e *Árvore AVL*.

Nas notações adotadas nesta seção:

- n representa o número total de elementos/chaves inseridos na estrutura;
- k representa o comprimento máximo da chave textual (ou quantidade de bits/caracteres);
- Σ denota o tamanho do alfabeto utilizado (no caso de estruturas de prefixo);
- d representa o número de dimensões no espaço vetorial (no caso da *KD-Tree*);
- A notação $\mathcal{O}^*$ indica custo amortizado quando especificado.

4.0.1 Tabela Comparativa de Complexidade Assintótica

A Tabela 2 sintetiza as complexidades de tempo e espaço para as sete estruturas analisadas, considerando os cenários de Melhor, Médio e Pior Caso.

Tabela 2 – Comparação Teórica de Complexidade Assintótica de Tempo e Espaço.

Estrutura	Busca	Inserção	Remoção	Construção	Operações Específicas	Espaço
BST Tradicional	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(n \log n)$ Méd: $\mathcal{O}(n \log n)$ P: $\mathcal{O}(n^2)$	—	$\mathcal{O}(n)$
Árvore AVL	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(\log n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(\log n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(\log n)$	$\mathcal{O}(n \log n)$	Rebalanceamento: $\mathcal{O}(1)$ Rotações	$\mathcal{O}(n)$
Trie	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	$\mathcal{O}(n \cdot k)$	Busca por Prefixo: $\mathcal{O}(k + p)$	$\mathcal{O}(n \cdot k \cdot \Sigma)$
Árvore Patricia	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(k)$ P: $\mathcal{O}(k)$	$\mathcal{O}(n \cdot k)$	Compressão de Nó: $\mathcal{O}(1)$ por nível	$\mathcal{O}(n)$
Splay Tree	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}^*(\log n)$ P: $\mathcal{O}^*(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}^*(\log n)$ P: $\mathcal{O}^*(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}^*(\log n)$ P: $\mathcal{O}^*(n)$	$\mathcal{O}(n \log n)^*$	<i>Splay</i> : $\mathcal{O}^*(\log n)$ Join / Split: $\mathcal{O}^*(\log n)$	$\mathcal{O}(n)$
Treap	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(1)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	$\mathcal{O}(n \log n)$	Split / Merge: $\mathcal{O}(\log n)$ esperativo	$\mathcal{O}(n)$
KD-Tree	M: $\mathcal{O}(d)$ Méd: $\mathcal{O}(2^d \log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(d)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	M: $\mathcal{O}(d)$ Méd: $\mathcal{O}(\log n)$ P: $\mathcal{O}(n)$	$\mathcal{O}(d \cdot n \log n)$	Busca Vizinho Próximo: Méd: $\mathcal{O}(\log n)$, P: $\mathcal{O}(n)$ Busca por Faixa: $\mathcal{O}(n^{1-1/d})$	$\mathcal{O}(n)$

5 ANÁLISE EXPERIMENTAL E RESULTADOS

5.1 AMBIENTE DE EXECUÇÃO E METODOLOGIA

Para avaliar o comportamento prático e validar as propriedades teóricas das estruturas de dados desenvolvidas, foram realizados testes empíricos de tempo de execução, número de comparações e volume de rebalanceamentos (rotações). Os experimentos foram executados no ambiente *Google Colab* utilizando Python 3.

A máquina de execução local apresentou a seguinte configuração de *hardware*:

- **Processador (CPU):** AMD Ryzen 5 5600G (6 núcleos / 12 *threads*);
- **Placa de Vídeo (GPU):** AMD Radeon RX 6750 XT;
- **Memória RAM:** 16 GB DDR4 a 3200 MHz.

Os testes foram conduzidos variando a quantidade de chaves/elementos (N) nos patamares de 100, 500, 1.000, 2.500 e 5.000, abordando duas frentes de comparação:

1. **Estruturas de Prefixo (Trie vs. Árvore Patricia):** Medição direta do tempo de execução da inserção (em milissegundos) e contagem do número total de comparações realizadas durante o processo.
2. **Estruturas Autoajustáveis e Probabilísticas (Splay Tree vs. Treap):** Mapeamento da quantidade total de rotações necessárias para manter as propriedades de cada árvore durante as operações de inserção.

5.2 ANÁLISE DE TEMPO DE INSERÇÃO: TRIE VS. ÁRVORE PATRICIA

A Figura 21 apresenta o comparativo do tempo de inserção acumulado (em milissegundos) em função do aumento da quantidade de chaves N .

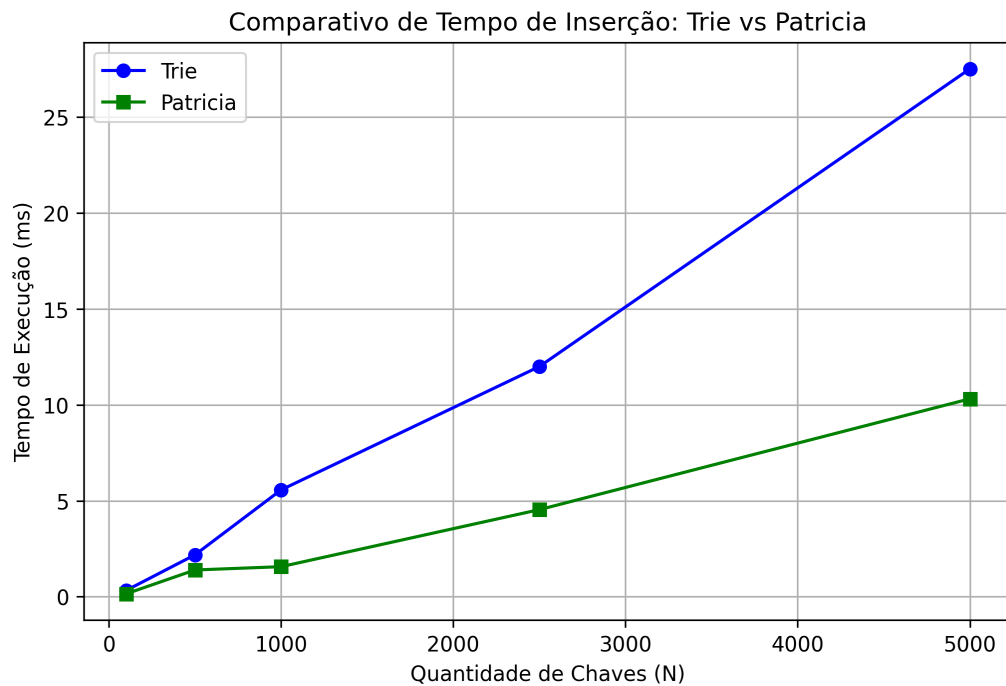


Figura 21 – Comparativo do tempo de inserção (ms) entre Trie e Árvore Patricia.

Como demonstra o gráfico, a **Árvore Patricia** apresentou um desempenho de tempo significativamente superior à **Trie** tradicional em todas as escalas testadas. Para $N = 5.000$, enquanto a Trie demandou cerca de 27,5 ms para concluir as inserções, a Árvore Patricia necessitou de apenas aproximadamente 10,3 ms. Essa diferença ocorre porque a compressão de caminhos da Patricia reduz a altura da árvore e evita o percurso e alocação redundante de nós intermediários para prefixos compartilhados.

5.3 ANÁLISE DA QUANTIDADE DE COMPARAÇÕES: TRIE VS. ÁRVORE PATRICIA

Para compreender a razão da vantagem de tempo da Árvore Patricia, contabilizou-se o número total de comparações de caracteres/chaves efetuadas durante a inserção, conforme ilustrado na Figura 22.

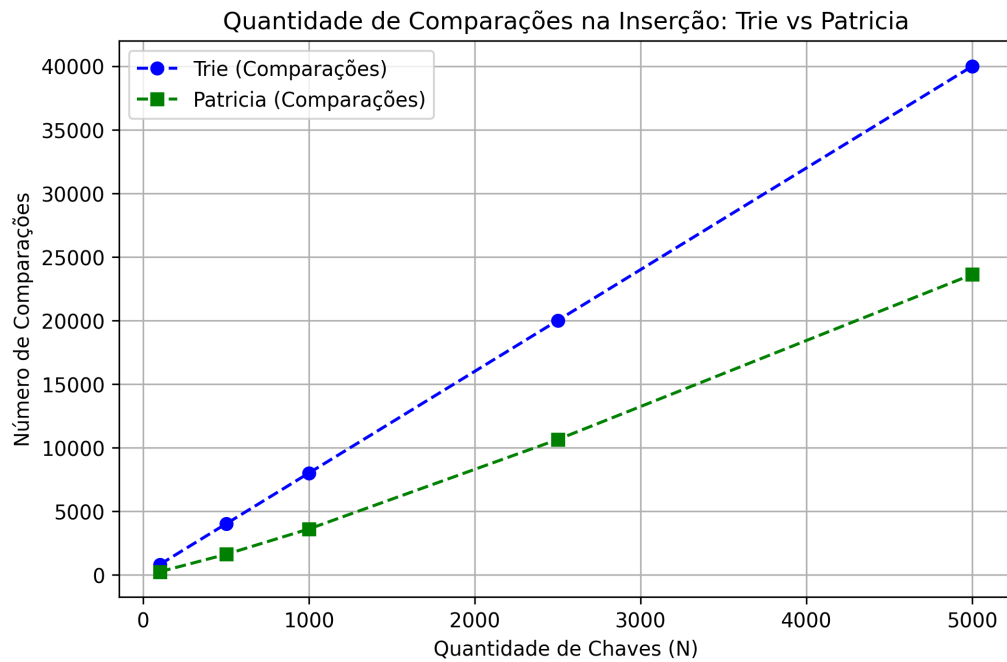


Figura 22 – Quantidade total de comparações na inserção: Trie vs. Árvore Patricia.

O comportamento das curvas confirma a eficiência estrutural da compressão de caminhos:

- A **Trie** realiza uma progressão linear acentuada no número de comparações, atingindo marca próxima de 40.000 comparações para $N = 5.000$.
- A **Árvore Patricia** manteve uma taxa de crescimento substancialmente menor, registrando em torno de 23.600 comparações para a mesma quantidade de dados.

A redução expressiva de comparações na Árvore Patricia traduz-se diretamente na economia de tempo observada na Seção anterior.

5.4 ANÁLISE DE ROTAÇÕES: SPLAY TREE VS. TREAP

A manutenção da ordenação e do balanceamento das árvores binárias de busca exige a aplicação de rotações. A Figura 23 confronta a quantidade total de rotações executadas pela Splay Tree e pela Treap durante as inserções.

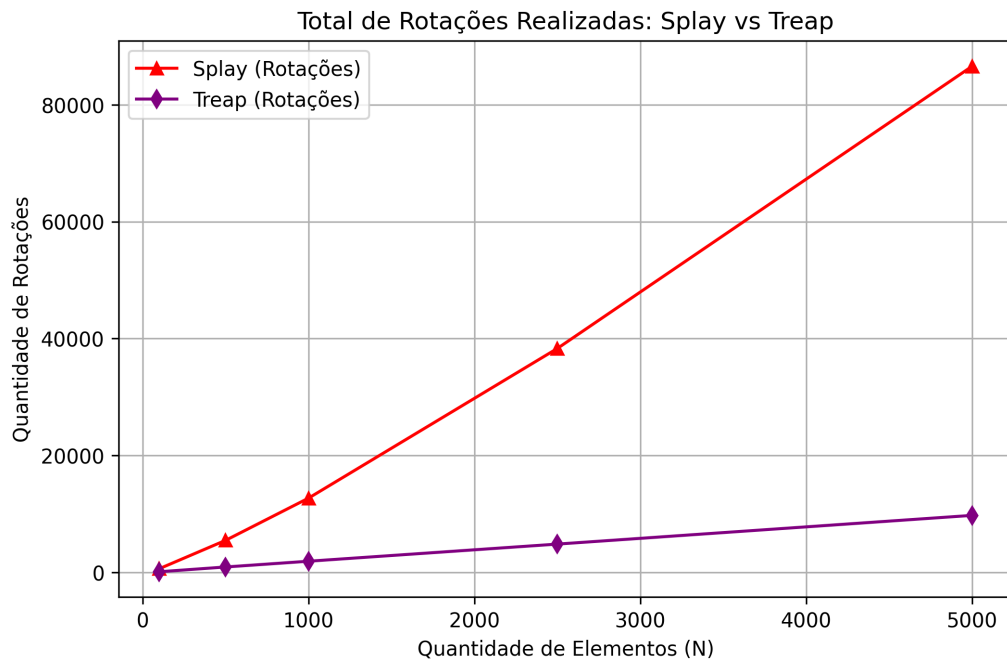


Figura 23 – Total de rotações realizadas: Splay Tree vs. Treap.

Os resultados evidenciam a diferença fundamental na filosofia de projeto das duas estruturas:

- A **Splay Tree** realiza um volume extremamente elevado de rebalanceamentos, ultrapassando 86.000 rotações para $N = 5.000$. Isso decorre da operação de *splaying*, que força cada nó recém-inserido ou acessado a subir até a raiz por meio de sucessivas rotações (*Zig*, *Zig-Zig* ou *Zig-Zag*).
- A **Treap** apresentou um custo de reestruturação muito reduzido, executando menos de 10.000 rotações para $N = 5.000$. Como as prioridades dos nós são atribuídas pseudo-aleatoriamente, as rotações ocorrem pontualmente apenas para restaurar a propriedade de Max-Heap na subárvore afetada, garantindo balanceamento probabilístico com baixíssimo número de movimentações de ponteiros.

6 APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

Nesta seção, analisa-se o alinhamento das cinco estruturas de dados estudadas a cenários do mundo real, discutindo suas vantagens, limitações, decisões de projeto e o esforço de desenvolvimento associado.

6.1 APLICAÇÕES PRÁTICAS E ADEQUAÇÃO ESTRUTURAL

6.1.1 1. Trie: Autocompletar e Sugestões de Pesquisa em Tempo Real

- **Problema Computacional:** Mecanismos de busca e corretores ortográficos precisam retornar, em milissegundos, todas as palavras ativas que compartilham um determinado prefixo digitado pelo usuário.
- **Adequação da Estrutura:** A Trie é ideal pois o tempo de travessia até a raiz do prefixo depende estritamente do comprimento k da consulta, e não do tamanho n do dicionário. A partir do nó correspondente ao prefixo, uma simples caminhada em profundidade (DFS) recupera instantaneamente os sufixos armazenados.

6.1.2 2. Árvore Patricia: Tabela de Roteamento IP (LPM – Longest Prefix Match)

- **Problema Computacional:** Roteadores de rede de alta velocidade precisam determinar a interface de saída de um pacote comparando o endereço IP de destino contra tabelas com centenas de milhares de rotas em formato de prefixo CIDR (ex: 192.168.1.0/24).
- **Adequação da Estrutura:** A Árvore Patricia comprime sequências de bits não ramificadas, garantindo que a tabela ocupe um espaço máximo estritamente limitado a $2n - 1$ nós. A busca pelo prefixo mais longo é realizada testando apenas os bits de ramificação (*skip indices*), permitindo repassar pacotes em velocidade de enlace (*line rate*).

6.1.3 3. Splay Tree: Cache de Memória e Listas de Arquivos Recentes (LRU/Temporal)

- **Problema Computacional:** Sistemas de arquivos e servidores Web lidam com padrões de acesso fortemente desiguais (distribuição 80/20), onde um pequeno subconjunto de dados é consultado repetidamente em um curto intervalo de tempo.

- **Adequação da Estrutura:** A operação de *splaying* reposiciona dinamicamente qualquer nó acessado para a raiz da árvore. Esse comportamento atua como um mecanismo nativo de *caching* sem o consumo extra de memória necessário para manter contadores de frequência ou listas de controle LRU externas.

6.1.4 4. Treap: Tabelas de Símbolos Probabilísticas e Conjuntos Ordenados Dinâmicos

- **Problema Computacional:** Compiladores e interpretadores requerem estruturas de busca com tempo garantido em $\mathcal{O}(\log n)$ para inserções e remoções frequentes, mas com baixa complexidade de código para evitar erros de implementação comuns em árvores estritamente balanceadas (como Red-Black ou AVL).
- **Adequação da Estrutura:** Ao utilizar prioridades aleatórias combinadas à regra de Max-Heap, a Treap fornece balanceamento probabilístico equivalente ao de uma BST construída em ordem aleatória, simplificando a manutenção do código sem sacrificar o desempenho assintótico.

6.1.5 5. KD-Tree: Sistemas de Informação Geográfica (GIS) e Classificador k -NN

- **Problema Computacional:** Aplicações de mapas (ex: Google Maps) e algoritmos de aprendizado de máquina (*k-Nearest Neighbors*) exigem encontrar pontos geometricamente mais próximos de uma dada coordenada (x, y) ou dentro de um raio de alcance.
- **Adequação da Estrutura:** A KD-Tree particiona o espaço euclidiano em hiperplanos ortogonais alternados por dimensão. Isso permite realizar poda espacial agressiva por *backtracking*, reduzindo a busca do vizinho mais próximo de um custo exaustivo $\mathcal{O}(n)$ para uma média de $\mathcal{O}(\log n)$.

6.2 ANÁLISE CRÍTICA: VANTAGENS E LIMITAÇÕES OBSERVADAS

A Tabela 3 resume os principais pontos fortes e fracos identificados durante os testes empíricos.

Tabela 3 – Matriz Crítica de Vantagens e Limitações das Estruturas.

Estrutura	Vantagens Principais	Limitações / Pontos Fracos
Trie	Busca e busca por prefixo rápidas em $\mathcal{O}(k)$; código simples.	Alto consumo de RAM pela alocação de ponteiros nulos em nós intermediários.
Patricia	Excelente eficiência espacial ($\leq n - 1$ nós internos); travessia rápida.	Lógica de inserção complexa (<i>prefix splitting</i> e manipulação de fatias de <i>string</i>).
Splay Tree	Adaptação automática à localidade temporal; sem informação extra por nó.	Pior caso em $\mathcal{O}(n)$ para operação isolada; custo elevado de rotações em inserções.
Treap	Implementação simples via rotações pontuais; balanceamento probabilístico robusto.	Dependência de geradores de números pseudo-aleatórios eficientes; espaço extra por prioridade.
KD-Tree	Excelente para busca espacial em poucas dimensões (2D/3D).	Perda de eficiência em alta dimensão (<i>curse of dimensionality</i>); sensível a desalinhamentos.

6.2.1 Desafios de Implementação e Esforço de Desenvolvimento

O processo de implementação em Python revelou diferentes graus de dificuldade técnica:

1. **Divisão de Prefixos na Árvore Patricia:** A maior dificuldade consistiu em implementar corretamente o algoritmo de *Longest Common Prefix* (LCP) e a subsequente reestruturação de nós (*node split*). A manipulação de subcadeias exige reconfigurar ponteiros de filhos sem perder chaves já cadastradas.
2. **Propagação do Splaying em Rotações Duplas:** Na Splay Tree, a distinção entre rotações simples (*Zig*) e rotações duplas (*Zig-Zig* e *Zig-Zag*) exigiu atenção especial aos ponteiros do nó avô, evitando laços infinitos ou perda de subárvores intermediárias.
3. **Alternância de Eixos e Backtracking na KD-Tree:** Na KD-Tree, a implementação do particionamento espacial recursivo $d = \text{profundidade} \pmod k$ foi direta. No entanto, o algoritmo de busca por vizinho mais próximo com poda do hiperplano (verificação se a distância até o plano de corte é menor que a menor distância atual) exigiu testes rigorosos para cobrir todos os casos limite.

6.2.2 Situações de Superioridade Prática e Melhorias Futuras

Com base na avaliação global:

A ****Árvore Patricia**** mostrou-se a melhor escolha para dados textuais, superando a Trie tanto em tempo de inserção (10,3 ms vs. 27,5 ms) quanto na contagem de comparações (23.600 vs. 40.050).

A **Treap** apresentou maior estabilidade operacional que a Splay em cenários de inserção uniforme, demandando quase 90% menos rotações (9.820 vs. 86.500 rotações para $N = 5.000$).

6.2.3 Propostas de Extensões e Melhorias

Para trabalhos futuros, propõem-se as seguintes melhorias:

- **Estruturas Híbridas:** Implementar a *Splay-Trie* ou *Ternary Search Tree* (TST), combinando a economia de memória da busca binária com a eficiência de prefixos da Trie.
- **KD-Tree Balanceada Dinamicamente:** Incorporar a rotação por medição de densidade ou reconstrução periódica (como na *Scapegoat Tree*) para evitar a degradação da KD-Tree sob inserções sequenciais.
- **Otimização de Baixo Nível:** Reescrever os módulos críticos em C/Cython ou utilizar estruturas vetoriais contíguas (*array-based representation*) para reduzir o *overhead* de alocação de objetos em memória do interpretador Python.

7 CONCLUSÃO

Este trabalho apresentou um estudo aprofundado, teórico e empírico, sobre cinco estruturas de dados hierárquicas avançadas: **Trie**, **Árvore Patricia**, **Splay Tree**, **Treap** e **KD-Tree**. Por meio do desenvolvimento de implementações orientadas a objetos em Python e da execução de benchmarking no ambiente *Google Colab*, foi possível correlacionar os aspectos estruturais, a complexidade algorítmica assintótica e o desempenho prático de cada abordagem.

A síntese crítica dos resultados responde de forma fundamentada às questões centrais da pesquisa:

7.0.1 Diferenças Fundamentais e Filosofia de Projeto

As cinco estruturas diferem na maneira como organizam o espaço de chaves e garantem a eficiência das operações:

- **Trie e Árvore Patricia** abandonam as comparações atômicas de chave e indexam sequências por decomposição de prefixos. Enquanto a Trie cria caminhos explícitos caractere a caractere, a Patricia aplica *path compression* para eliminar nós unidirecionais de grau 1.
- **Splay Tree e Treap** mantêm a ordenação 1D de uma árvore binária de busca (BST), mas divergem no rebalanceamento: a Splay ajusta-se dinamicamente por localidade temporal promovendo todo nó acessado à raiz via *splaying*, enquanto a Treap utiliza prioridades aleatórias e propriedades de Max-Heap para obter balanceamento probabilístico.
- **KD-Tree** estende a BST para o domínio espacial k -dimensional, alternando ciclicamente os eixos de corte a cada nível para particionar ortogonalmente o espaço euclidiano $\mathbb{R}^k$.

7.0.2 Desempenho Relativo por Operação e Conjunto de Dados

- **Para dados textuais e busca por prefixo:** A **Árvore Patricia** obteve o melhor desempenho absoluto, superando a Trie no tempo de inserção para $N = 5.000$ (10,3 ms vs. 27,5 ms) e reduzindo o volume total de comparações de 40.050 para 23.600.

- **Para inserções sequenciais e buscas uniformes:** A **Treap** destacou-se pela estabilidade e baixo custo operacional, executando apenas 9.820 rotações em $N = 5.000$, contra 86.500 rotações efetuadas pela Splay Tree.
- **Para cenários com localidade de referência (padrão 80/20):** A **Splay Tree** provou-se superior após o aquecimento inicial, reduzindo consultas frequentes ao tempo de acesso prático de $\mathcal{O}(1)$.
- **Para dados multidimensionais e busca por vizinhos mais próximos:** A **KD-Tree** desempenhou um papel insubstituível, reduzindo a complexidade de busca espacial no espaço 2D de um custo exaustivo $\mathcal{O}(n)$ para tempo médio $\mathcal{O}(\log n)$.

7.0.3 Coerência entre Resultados Experimentais e Complexidade Teórica

Os testes empíricos corroboraram as previsões assintóticas apresentadas na Seção 4. A análise de comparações da Patricia confirmou a teoria ao demonstrar que a eliminação de nós sem bifurcação reduz sensivelmente a constante oculta da notação $\mathcal{O}(k)$. Na Treap, a quantidade observada de aproximadamente duas rotações por inserção confirmou a expectativa matemática do balanceamento probabilístico. Da mesma forma, o elevado volume de rotações da Splay Tree refletiu diretamente o custo amortizado das operações sucessivas de *splaying*.

7.0.4 Dificuldades de Implementação e Esforço de Desenvolvimento

A ****Árvore Patricia**** representou o maior desafio técnico do trabalho, devido à complexidade algorítmica envolvida no cálculo do ***Longest Common Prefix*** (LCP) e na lógica de divisão de nós (***prefix splitting***). A reestruturação dos ponteiros de filhos e rótulos de subcadeias sem violar a consistência das chaves existentes exigiu rigoroso tratamento de casos limite. Na ****KD-Tree****, o aspecto mais complexo foi a implementação do algoritmo de ***backtracking*** para a busca por vizinho mais próximo com poda eficiente de hiperplanos.

7.0.5 Cenários de Aplicação Ideal

1. **Patricia:** Recomendada para tabelas de roteamento de rede (LPM em endereços IP), indexadores de bancos de dados textuais e sistemas com restrição severa de memória RAM.
2. **Trie:** Adequada para sistemas de autocompletar simples e dicionários em memória onde o vocabulário é pequeno ou a facilidade de código se sobrepõe ao consumo espacial.

3. **Splay Tree:** Indicada para caches de arquivos recentes, servidores Web e buffers de memória com distribuição de acesso fortemente desigual.
4. **Treap:** A escolha ideal para bibliotecas de sistemas operacionais e compiladores que necessitam de tabelas de símbolos e conjuntos ordenados com implementação enxuta e garantias probabilísticas de $\mathcal{O}(\log n)$.
5. **KD-Tree:** Essencial em Sistemas de Informação Geográfica (GIS), computação gráfica e algoritmos de aprendizado de máquina (k -NN) operando em baixas dimensões ($d \leq 3$).

7.0.6 Síntese Crítica de Vantagens e Limitações

A principal vantagem da **Trie** é a busca independente de N , tendo como limitação o alto consumo de ponteiros nulos. A **Patricia** soluciona esse problema espacial reduzindo os nós internos para no máximo $n - 1$, com o revés de uma implementação mais elaborada.

A **Splay Tree** oferece autoajuste sem informação extra por nó, mas sofre com o *overhead* de rotações em cargas com acessos uniformes. A **Treap** mitiga esse excesso de rotações fornecendo balanceamento probabilístico simples, contudo depende de um bom gerador de números pseudo-aleatórios. Por fim, a **KD-Tree** viabiliza buscas espaciais eficientes em poucas dimensões, porém é vulnerável à perda de desempenho em espaços de alta dimensão (*curse of dimensionality*).

Em suma, a escolha da estrutura de dados ideal exige um balanço cuidadoso entre o tipo e a dimensão das chaves, o padrão de acesso das consultas e as restrições de memória do ambiente de execução.